Formalization Notes: Trust Boundary, Cost Model, Statement Conventions, and Paper Correspondence (BMMGrid)

Paper version: paper.tex, 2026-10-25, md5 f3e6ff31cfeb9fedac85b3af9ef2f9db.

This document records the formalization architecture, paper-to-Lean theorem crosswalk, and the four classes of formalization notes for the Lean 4 proof of "Combinatorial Boolean Matrix Multiplication in $n^3/2^{\Omega(\sqrt{\log n})}$ Time" (BMMGrid):

- **Section 1 — Paper-to-Lean Theorem & Algorithm Crosswalk**: Mapping between the theorems, lemmas, corollaries, and algorithms of paper.tex and their Lean 4 declarations, explicitly distinguishing structural/accuracy theorems from CostM complexity theorems.
- **Class A (§2) — Assumptions (Trust Boundary)**: The two external cited-sampler hypotheses (Stage1Sampler, Stage2Sampler), their correspondence with §3.3 Lemma 3.3 (\label{lem:two-stage-sampler}) and Appendix B (\label{app:sampler}), and their transcription notes vs. the printed literature theorems.
- **Class B (§3) — Cost Model**: The two-tier CostM declared-charge model, representation assumptions, multi-precision pricing, the 107-site free-payload combinator table (self-generated by lean/scripts/gen_payload_table.py; 106 on the headline path, 1 off-path), the AST/unused-declaration linters and headline-cone exclusion scope, and the relation to RAM models.
- **Class C (§4) — Statement Conventions**: Separation of structural and CostM theorems (including thm_main_decomp, thm_main_decomp_of_samplers, thm_2d_decomp_concrete, and thm_2d_decomp_uncapped_concrete), the two 2D decomposition implementations (Decomp2D.lean vs. DyadicBucketingAndGoodCube.lean), $0/0 = 0$, structural recursion fuel counters (rRectFuel, rRect2DFuel, rUncappedRect2DFuel, rCubeFuel), the $\mathbb{R}$ / Finset interface (noncomputable headline *Tracked functions and uniform EstimateNorm accuracy via accExp), the floor-division identity triDetParamD_eq_floor_div, ValidCubePiece empty-subset totalization and entrywise submatrix dominance (cappedABDecomposition_dominance) vs. §1.2 Definition 1.1 (\label{def:pieces}) & Theorem 1.1 (\label{thm:main-decomp}), and non-formalized discussion/consequence items.
- **Class D (§5) — Implementation Choices Preserving the Proved Guarantees**: Six implementation choices (uniform α' parameter, fuel totalization, EstimateNorm cost envelope in Sift, full-list fold in Algorithm 6, single-witness clearing per detector call in the BMM reduction, and $O(|U||V| + |U|^2)$ declared charge in Prune) that preserve the proved guarantees and asymptotic complexity bounds.

1 Paper-to-Lean Theorem & Algorithm Crosswalk

Paper Item	Structural / Correctness Theorem(s)	Time / CostM Bound Theorem(s)	Notes & Scope
§1.2 Def. 1.1 (\label{def:pieces}) & Thm. 1.1 (\label{thm:main-decomp})	Decomp3D.lean: thm_main_decomp, thm_main_decomp_of_samplers, thm_3d_decomp_part2, cappedABDecomposition_dominance	TriangleAndBMM.lean: cappedABDecomposition_concrete_bounds, thm_main_3d_decomposition	thm_main_decomp and thm_main_decomp_of_samplers prove structural, entrywise dominance ($A_k \le A[X_k, Y_k], B_k \le B[Y_k, Z_k]$), piece-count, and $C_{1/4,1/4}$ potential bounds for all $0 < \varepsilon \le 1/160$ from Stage1Sampler/Stage2Sampler; closed-form $[n]^3$ time bound proved at $\varepsilon = 1/160$.

Paper Item	Structural / Correctness Theorem(s)	Time / CostM Bound Theorem(s)	Notes & Scope
§1.2 Thm. 1.2 (\label{thm:main-bmm})	TriangleAndBMM.lean: computableTriangleDetection_correct, computableBMM_correct, hasTriangle_eq_true_iff_strictMono_of_symm_irrefl	TriangleAndBMM.lean: thm_main_triangle_detection, thm_main_triangle_detection_graph, thm_main_bmm (CostM); thm_main_triangle_detection_log_cost_absorption, thm_main_bmm_log_cost_absorption (aliases *_log_cost)	Formalizes unit-cost CostM step count t and Cook–Reckhow log-cost absorption $t \cdot 12(\lfloor \log_2 n \rfloor + 5)^2 = t \cdot 12(\texttt{bitLengthNat}\ n + 4)^2$. hasTriangle is tripartite; hasTriangle_eq_true_iff_strictMono_of_symm_irrefl bridges symmetric irreflexive Adj to $i < j < k$.
§3.2 Prop. 3.1 (\label{prop:regular-product})	KLMUniformity.lean: prop_regular_matrices_uniform_product	N/A (structural discrepancy/uniformity theorem)	KLM grid-regular product uniformity bound.
§3.3 Alg. 1 (Prune) & Lem. 3.2 (\label{lem:min-degree})	MinDegreeAndDyadicSift.lean: lem_min_degree, minDegreeLoop_spec	MinDegreeAndDyadicSift.lean: minDegreeTracked_snd_le	Exports top-level $\lvert U' \rvert \geq \max(1, \lfloor (1-\gamma_0)\lvert U \rvert \rfloor)$, Case 1 ($\varepsilon_0$-min-degree), and Case 2 ($\lvert U' \rvert = \lfloor (1-\gamma_0)\lvert U \rvert \rfloor$, $\lvert U \setminus U' \rvert = \lceil \gamma_0 \lvert U \rvert \rceil$, and density boost); CostM bound proved in minDegreeTracked_snd_le.
§3.3 Lem. 3.3 (\label{lem:two-stage-sampler}) & App. B (\label{app:sampler})	CitedResults.lean: Stage1Sampler, Stage2Sampler; ObliviousSampler.lean: stage2FinM_sampler, twoStage_compose_general, twoStage_compose	ObliviousSampler.lean: twoStageConstructTracked_snd_le, TwoStageSamplerOracle	Stage1Sampler and Stage2Sampler are the sole external literature assumptions; $\{0,1\}^m \to \mathrm{Fin}\, M$ and two-stage composition are machine-checked (construction cost stated as $(\lvert Y \rvert + 1)^2 \cdot F \cdot B$; Lemma 3.3's $\lvert Y \rvert \cdot$ polylog factor is relaxed).
§4.1 Alg. 2 (EstimateNorm) & Lem. 4.1 (\label{lem:approx-grid-norm})	ApproxGridNorm.lean: lem_approx_grid_norm, lem_approx_grid_norm_seq_real	MinDegreeAndDyadicSift.lean: concreteApproxGridNormVTracked_snd_le	lem_approx_grid_norm (IsObliviousSampler) and lem_approx_grid_norm_seq_real (IsSeqSampler) prove the accuracy bound; CostM time bound proved in concreteApproxGridNormVTracked_snd_le.
§4.2 Alg. 3 (Sift) & Lem. 4.2 (\label{lem:dyadic-sift})	MinDegreeAndDyadicSift.lean: lem_dyadic_sift, lem_dyadic_sift_concrete	MinDegreeAndDyadicSift.lean: dyadicSiftTracked_snd_le, dyadicSiftConcreteTracked_snd_le	Case A / Case B / regular certification and CostM bound.
§4.3 Alg. 4 (FindRect, Decompose2D), Lem. 4.3, Thm. 4.4, Cor. 4.5 & Lem. 4.6	Decomp2D.lean: thm_2d_decomp_concrete, thm_2d_decomp (capped), concreteOracleV_isValid, rRect2DFuel_width_le_paper, thm_2d_decomp_uncapped, thm_2d_decomp_uncapped_concrete (uncapped, all $D \geq 2$), thm_2d_decomp_riemann_telescoping; DyadicBucketingAndGoodCube.lean: cappedGoodRect3D_spec, cappedADecomp3DLoop_pieces_spec, cappedADecomp3D_eq_cappedADecompositionConcrete; Decomp3DPotential.lean: cappedADecomp3DLoop_scale_and_residual_spec	Decomp2D.lean: thm_2d_decomp_concrete, thm_2d_decomp_uncapped_concrete, cappedGoodRectConcreteTracked_snd_le, cappedADecompositionConcreteTracked_snd_le; DyadicBucketingAndGoodCube.lean: cappedGoodRect3DLoopTracked_snd_le, cappedADecomp3DLoopTracked_snd_le	thm_2d_decomp_concrete proves an artifact-constant version of Theorem 4.4 (partition, zero-extension, residual $X_{\mathrm{sub}} = X \wedge Y_{\mathrm{sub}} = Y$ and density $\leq \tau_{\mathrm{low}}$, strict $\tau_{\mathrm{low}} < \mathbb{E}[A_\ell]$ and scale $s_\ell \leq D$ for non-residual pieces, column-width, piece-count, L_s and L_{reg} bounds, Clause 3 mass conservation, area-shrinkage, telescoping, and symbolic CostM time) for concreteOracleV S; thm_2d_decomp_uncapped and thm_2d_decomp_uncapped_concrete prove Corollary 4.5 for all $D \geq 2$.
§4.4 Prop. 4.7 (\label{prop:tightness})	Decomp2D.lean: rem_obstacles_single_step_tightness, rem_obstacles_multiblock_sift_tightness	N/A	Single-step $2^{-\lfloor d^2/2 \rfloor}$ sifting tightness witness conditional on supplied parameters / PlantedMultiBlockConfig; configuration existence and multi-round tree lower bound not formalized.
§5.1–5.5 Alg. 5 (FindCube, Decompose3D) & Lems. 5.1, 5.3–5.5, 5.9 & Claims 5.6–5.8	DyadicBucketingAndGoodCube.lean: lem_dyadic_col_degree_bucketing, cappedGoodCubeLoop_spec, thm_3d_decomp_part1; Decomp3DPotential.lean: table1_budget_cinco_sum_le, T3DTree_jump_le, step_T3DTree_close; Decomp3D.lean: thm_3d_decomp_part2	Decomp3D.lean: cappedABDecompositionTracked_snd_le; TriangleAndBMM.lean: cappedABDecomposition_concrete_bounds	3D decomposition (including both directions of Algorithm 5 Line 5's residual slice assignment (\label{line:cube-zl}), strict $\alpha_{\mathrm{low}} < \mathbb{E}[A_\ell]$, and $\beta^* < 16(1+\varepsilon\gamma/4)\beta_{\mathrm{init}} < 17\beta_{\mathrm{init}}$), 5-term Table 6 potential budget, jump-adaptive recurrence, and CostM complexity bound.
§6 Alg. 6 (TriangleDetect) & Lem. 6.1 (\label{lem:bmm-to-tridet})	TriangleAndBMM.lean: detectOnPiece_eq_true_iff, bmmFromTriDet_correct, nnz_C_strictly_decreases_on_triangle	TriangleAndBMM.lean: computableTriangleDetectionTracked_spec, lem_bmm_to_tridet_time_bound, computableBMMTracked_spec	Triangle detection and $m = \max(1, \lfloor n^{1/4} \rfloor)$ BMM reduction.

Paper Item	Structural / Correctness Theorem(s)	Time / CostM Bound Theorem(s)	Notes & Scope
App. C (\label{app:models})	ArithRAM.lean: ramInitAndAdd, ramReadMatrixEntry, ramWriteMatrixEntry, ramCheckTriangleTriple, ramCountLoop	TriangleAndBMM.lean: thm_main_triangle_detection_log_cost_absorption, thm_main_bmm_log_cost_absorption	Formalizes register-addressed ArithRAM and 4 standalone 2-to-6-instruction micro-kernels + 1 counter loop; Algorithms 1–6 are not compiled to an ArithRAM.

2 Class A — Assumptions (Trust Boundary; BMMGrid/CitedResults.lean is Authoritative)

External Sampler Hypotheses (BMMGrid/CitedResults.lean, primary-source excerpts in lean/SOURCES.md; §3.3 Lemma 3.3 \label{lem:two-stage-sampler} & Appendix B \label{app:sampler}). The headline theorems (thm_main_decomp_of_samplers, thm_main_triangle_detection, thm_main_triangle_detection_graph, thm_main_bmm; and thm_main_3d_decomposition via the TwoStageSamplerOracle they discharge) take two external literature sampler schemas over sample sequences with repeats (List (List (Fin N)) and List (List (Fin (2 ^ m))), sequence average expectSeq, and two-sided accuracy predicate IsSeqSampler):

1. Stage1Sampler ([AFKLM24, Lemma 4.4, arXiv:2311.09095v2, p. 17] = [GW97, STOC 1994, pp. 574-583; Random Structures & Algorithms 11(4):315-343, 1997, Section 3]): $(1/e, 1/\delta)$-oblivious sampler on Fin N with:
   - Uniform sequence length: len N e $\delta \le c \cdot (e \cdot \delta + 1)^k$.
   - Family size: (fam N e δ).length $\le N \cdot c \cdot (e \cdot \delta + 1)^k$.
   - CostM construction cost (for $1 \le N$): (famTracked N e δ).2 $\le N \cdot c \cdot (e \cdot \delta \cdot (\text{Nat.log 2 N} + 1) + 1)^k$. Here the $1 \le N$ guard avoids the $N = 0$ degenerate case where (famTracked 0 e δ).2 ≥ 1 for any combinator-built CostM definition, and the (Nat.log 2 N + 1) factor in the polynomial base generously absorbs the poly$(\log N, e, \delta)$ per-step expander-walk and arithmetic overhead of [GW97] / [AFKLM24, Lemma 4.4].
2. Stage2Sampler ([GUV09, Theorem 1.5 (and Theorem 5.12), JACM 56(4), Article 20, pp. 20:1-20:34 (p. 4 / Article p. 20:4), 2009] + [Vadhan 2012, Corollary 6.24(1), printed p. 186 / PDF p. 189]; Appendix B also cites [Zuckerman 1997, Lemma 2.6 & Proposition 2.7] for the extractor-to-sampler equivalence and [Zuckerman 1997, Theorem 3.17, p. 15] for historical explicit-sampler context): $(1/e, 1/\delta)$-averaging sampler on Fin (2 ^ m) ($\cong \{0,1\}^m$) with:
   - Uniform sequence length (logarithmic in δ): len m e $\delta \le c \cdot ((m+1) \cdot e \cdot (\text{Nat.log 2 } \delta + 1))^k$.
   - Family size: (fam m e δ).length $\le c \cdot (2^m \cdot e \cdot \delta + 1)^k$.
   - CostM construction cost: (famTracked m e δ).2 $\le c \cdot (2^m \cdot e \cdot \delta + 1)^k$. Here the single existential exponent $k \in \mathbb{N}$ absorbs the $O(1/\alpha)$ constant exponent from [GUV09, Theorem 1.5] (the extractor-based sampler has $2^{O(m+\log \delta)}$ seeds and poly$(m, e, \log \delta)$ samples, so its construction cost is poly$(2^m, e, \delta)$).
3. **Primary-Source Verification**: Verbatim primary-source quotations and the 4-line derivation of the Stage2Sampler bounds from [GUV09, Theorem 1.5] + [Vadhan 2012, Corollary 6.24(1)] are recorded in the module header of BMMGrid/CitedResults.lean; direct PDF fetch URLs and SHA-256 digests are recorded in lean/SOURCES.md.

Transcription Notes ((i)–(v), matching BMMGrid/CitedResults.lean):

- *(i) Uniform sequence length (Stage1Sampler.len_eq)*: [AFKLM24, Lemma 4.4] states subset bounds $|T| \le$ poly$(\varepsilon^{-1}, \delta^{-1})$ without asserting equal lengths across $T \in \mathcal{T}$. Uniform length holds both for the underlying [GW97] fixed-length random-walk construction and by Appendix B's generic reduction (running at accuracy $\varepsilon/4$ and padding each subset to a common power-of-two length $N_1 = 2^p$), and is required for the union-bound composition (composeFamilies_isSeqSampler) and the common-denominator identity in ApproxGridNorm.
- *(ii) Power-of-two Stage-2 domain (Fin (2 ^ m) → Fin M, proved)*: Stage2Sampler is stated over $\{0,1\}^m \cong$ Fin(2^m); the modulo-projection reduction to arbitrary Fin M is machine-checked in Lean (stage2FinM_sampler in BMMGrid/ObliviousSampler.lean).

- *(iii) Cost bound transcription (`cost_le`)*: The sources' polynomial deterministic RAM construction-time claims are transcribed as polynomial `CostM` upper bounds with existential constants $c, k \in \mathbb{N}$ (with $1 \leq N$ on `Stage1Sampler.cost_le` so `cost_le` does not force `(famTracked 0 e δ).2 = 0`); no concrete constant values are claimed.
- *(iv) Conditional trust boundary (no inhabitant)*: No Lean inhabitant of `Stage1Sampler` or `Stage2Sampler` is provided (the $n = 2$ examples in `BMMGrid.lean` are conditional sanity checks parameterized by `S1 : Stage1Sampler` and `S2 : Stage2Sampler`; trivial exhaustive or product-space Chebyshev families do not satisfy the joint linear-in-N family-size, N-free sample-length, and poly($\log \delta$) sample-length bounds, so any inhabitant amounts to formalizing `[GW97]`/`[GUV09]`, which is out of scope); the headline theorems are conditional on the cited results as transcribed above.
- *(v) One-sided vs. two-sided sampler error*: `[Vadhan 2012, Definition 3.29, printed p. 71 / PDF p. 74]` defines averaging samplers with a one-sided bound; the two-sided `IsSeqSampler` bound follows "up to a factor of 2 in the failure probability δ" (Vadhan, p. 71, via $1 - f$), absorbed into c_2, k_2.

Machine-Checked Two-Stage Composition (`BMMGrid/ObliviousSampler.lean` ↔ §3.3 Lemma 3.3 `\label{lem:two-stage-sampler}` & Appendix B `\label{app:sampler}`):

- The composition of `S1 : Stage1Sampler` and `S2 : Stage2Sampler` into an (ε, δ)-oblivious sequence sampler with linear family size in $|Y|$ and logarithmic sequence length in $1/\delta$ (`twoStage_compose`, `twoStage_compose_general`, `twoStageConstructTracked`) is machine-checked in Lean; nothing in the composition is assumed.
- Set-based `IsObliviousSampler` (`Defs.lean`), unconditional Chebyshev tuple-sampler constructions (`constructChebyshevTupleSampler`, `lem_approx_grid_norm_chebyshev_tuple`, `lem_dyadic_sift_chebyshev` in `ObliviousSampler.lean`, `ApproxGridNorm.lean`, and `MinDegreeAndDyadicSift.lean`; correctness only, with $|\Omega| = |Y|^k$), and the explicit small-universe oracle inhabitant `smallTwoStageSamplerOracle` / `twoStageSamplerOracle_nonempty_of_le` (`ObliviousSampler.lean`, providing an unconditional inhabitant of `TwoStageSamplerOracle (Fin n) (1/160)` for all $n \leq 5{,}566{,}362{,}550{,}596$) are documented on the `auxiliaryRoots` allowlist outside the headline dependency cone.

3 Class B — Cost Model (`CostM` Two-Tier Charges & 107-Site Payload Table)

Declared-Charge Monad (`CostM` α `:=` $\alpha \times \mathbb{N}$, `BMMGrid/CostMonadAndLinter.lean`). Algorithms 1–6 (`Prune`, `EstimateNorm`, `Sift`, `FindRect` / `Decompose2D`, `FindCube` / `Decompose3D`, `TriangleDetect`, plus the §6 Lemma 6.1 BMM reduction) are formalized as pure total functions in `CostM` $\alpha := \alpha \times \mathbb{N}$, paired with untracked specifications and `.1` projection equivalences (`fooTracked_fst : (fooTracked ...).1 = foo ...`). Cost charges `.2` are **declared combinator charges** (not step counts of a compiled machine binary) organized in two tiers:

1. **Tier A — Bit-Level Tracked Arithmetic & List/Table Kernels**:
   - *§4.1 Algorithm 2 (`EstimateNorm`, `\label{alg:estimate-norm}`, `ApproxGridNorm.lean`)*: Two-stage sampler family construction (`twoStageConstructTracked` in `ObliviousSampler.lean` via `modFinFamTracked` / `modFinTracked`, `mapListTracked`, `flatMapListTracked`, and `lookupSeqTracked`), d-tuple intersection table construction (`buildTupleTableFinsetTracked` via `sumFinsetTracked` and `prodTupleBoolTracked`), and per-row table contraction and common-denominator assembly (`tabulateFinsetTracked` + `approxGridNormVxRowPairTracked` + `contractTupleTableFamilySimpleTracked` + `powNatTracked` + `mulNatTracked`).
   - *Multi-Precision Bit Pricing*: On the headline path, all multi-precision integer multiplications, powers, divisions, remainders, logarithms, square roots, and cross-multiplied comparisons on $O(d \log n)$-bit numbers are routed through the bit-priced primitives (`addNatTracked`, `subNatTracked`, `mulNatTracked`, `divNatTracked`, `modNatTracked`, `cmpLeNatTracked`, `cmpLtNatTracked`, `powNatTracked`, `log2NatTracked`, `sqrtNatTracked`, `cmpRatCrossLeTracked`, and `cmpRatCrossLtTracked`, charging `bitLengthNat` a + `bitLengthNat` b + 1 for `addNatTracked`/`subNatTracked`/`cmpLeNatTracked`/`cmpLtNatTracked`, `bitLengthNat` $a \cdot$ `bitLengthNat` $b + 1$ for `mulNatTracked`/`divNatTracked`/`modNatTracked`, $(d+1)(d \cdot$ `bitLengthNat` $a{+}1)^2$ for `powNatTracked`, `bitLengthNat` $a{+}1$ for `log2NatTracked`, (`bitLengthNat` $a{+}1)^2$ for `sqrtNatTracked`, and two `mulNatTracked` plus one `cmpLeNatTracked`/`cmpLtNatTracked`

for `cmpRatCrossLeTracked`/`cmpRatCrossLtTracked`); the two table-contraction accumulators in `contractTupleTableSimpleTracked` and `contractTupleTableFamilySimpleTracked` (`ApproxGridNorm.lean:1419, 1477`, rows 11–12) add an $O(d \log B + \log |X|)$-bit counter per table entry at unit charge, an undercharge of $O(d \log B + \log n)$ bit operations per entry. The (x, T) contraction loop executes $|X| \cdot |\mathcal{T}| \le |X|(|Y|+1)F$ times, so bit-pricing these additions would multiply the contraction phase by $\Theta(\log n)$, which the n-independent quadratic envelope of `concreteApproxGridNormVTracked_snd_le` (and hence `thm_main_decomp`) cannot absorb. The unit charge is the word-RAM cost: with $\Theta(\log n)$-bit words the counter occupies $O(1 + d \log B / \log n)$ words—$O(1)$ for the pipeline parameters ($d = \Theta(\sqrt{\log n})$, $\log B = O(d)$) and at most $O(d \log B)$ in general, an n-independent factor not reflected in the declared charge; in the log-cost corollaries the $O(\log n)$-bit additions are covered by the per-step $12(\texttt{bitLengthNat}\ n + 4)^2$ multiplier. Besides these accumulators, the following headline-path steps are charged at unit cost or performed as uncharged arguments, each dominated by an adjacent charged scan or by an n-independent factor: index projections `indexFinset` and `getD` inside `transportFamTracked` and `lookupSeqTracked` (`ObliviousSampler.lean:551, 630`, rows 1–2, unit-charged via `retTracked` per element; the modulo projection `modFinTracked` at `ObliviousSampler.lean:1045` is bit-priced via `modNatTracked`), parameter expressions `stage2BitLen M e` (`ObliviousSampler.lean:1283`), `2 ^ logInvDelta`, and `(fam1_idx.headD []).length` (`ObliviousSampler.lean:1767, 1770`, dominated by `transportFamTracked`'s $\ge 2M + 2$ charge per sequence), `sumFinsetTracked X (prodTupleBoolTracked ...)` in `buildTupleTableFinsetTracked` (`ApproxGridNorm.lean:1388`, row 9, summing $\{0, 1\}$ indicators up to $|X|$), `selectWitnessRow` inside `scanFinsetTracked X` in `dyadicSiftTracked` (`MinDegreeAndDyadicSift.lean:1036`, row 21, at charge $|X| + 1$ so `dyadicSiftConcreteTracked_fst` remains unconditional), Base Case 2 support-list preparation `suppBList Y Z B`, `(Y.card * Z.card) / 2 ^ D`, and `splitListChunks` in `baseCase2DecompositionTracked` (`Decomp3DPotential.lean:859-866`, rows 68–69, dominated by the adjacent $\ge |X||Y| + |Y||Z| + 1$ `scanMatrixPairTracked` charges), and `blockSubTriplesList m` in `findBlockTriangleTracked` (`TriangleAndBMM.lean:3136`, row 100, dominated by the adjacent $3m^2 + 1$ `scanBlockExtractTracked` charge and $\le m^3 + 1$ `findSomeListByTracked` bound). Remaining unpriced pure expressions on the headline path consist only of index, tuple, and block enumerations (`List.range`, `piList T_list d` at `ApproxGridNorm.lean:1389`, `blockTriplesList` at `TriangleAndBMM.lean:3628`), the terminal accumulator reversal `acc.reverse` (`Decomp2D.lean:418, 422`), parameter scalings (`2 ^ accExp` and `d * accExp` at `MinDegreeAndDyadicSift.lean:2034`, `2 ^ j * p_ell` at `Decomp3D.lean:338`) dominated by their enclosing `mapListTracked`, `fold*`, or recursive calls, $O(1)$ word operations on cardinalities, the real-valued threshold parameters covered by the representation assumptions below, and the support and neighborhood filters (`suppA`, `suppB`, `S_k`, `N_A`, `N_B` in `detectOnPieceNatTracked`) already paid for by the preceding `scanMatrixPairTracked` / `scanMatrixTracked` charges.

- *Jump-Adaptive Depth Increments, Support-Count Scans, Cross-Multiplications & Fourth-Root Block Size*: Exact integer cross-multiplication and floor-log evaluation in `rawDeltaHBNatTracked` / `clampDeltaHBTracked` and `rawDeltaHANatTracked` / `clampDeltaHATracked` (`Decomp3DPotential.lean`), paired in `sStarLoopTracked` (`Decomp3D.lean`) with three explicit `scanMatrixTracked` support-count charges (`supportCountRealNat cubeRes.Y_star cubeRes.Z_star B`, `supportCountRealNat Y Z B`, `supportCountRealNat sl.X_ell sl.Y_ell sl.A_ell`) and `mapListTracked directSlices (fun sl => scanMatrixTracked sl.Y_ell sl.Z_ell (directSlicePiece B D sl))` for direct-slice emission, exact rational cross-multiplied comparisons (`cmpRatCrossLeTracked`, `cmpRatCrossLtTracked`, `cmpLeNatTracked`), and the iterated integer square root `blockSizeBMMTracked` (`TriangleAndBMM.lean`, computing $m = \max(1, \lfloor \sqrt{\lfloor \sqrt{n} \rfloor} \rfloor) = \max(1, \lfloor n^{1/4} \rfloor)$ via two `sqrtNatTracked` calls and `cmpLeNatTracked`, with cost $\le 24n^2$ in `blockSizeBMMTracked_snd_le`).

2. **Tier B — Combinators Carrying a Pure Value Payload (107 Definition-Level Call Sites Across 8 Modules)**: Across `BMMGrid/*.lean` (outside `CostMonadAndLinter.lean`), twelve `CostM` combinators construct a `CostM` value whose `.1` component is a pure Lean expression and whose `.2` component is a declared cost formula:

   - *Size-charge scan combinators (**51 definition-level sites**, plus 12 statement/proof/docstring references = 63 grep matches)*: `scanFinsetTracked U val := (val, U.card + 1)` (4 sites), `scanMatrixTracked U V val := (val, U.card * V.card + 1)` (32 sites), and `scanMatrixPairTracked U V W val := (val, U.card * V.card + V.card * W.card + 1)` (15 sites).
   - *Unit-cost return combinator (**45 definition-level sites**)*: `retTracked val := (val, 1)`, packaging already-

bound values or paired with an explicit `scan*Tracked` charge in the same step.

- *Per-element / sum / block-extract combinators (**11 definition-level sites**)*: `filterFinsetByTracked` (1 site, charging $|S| + 1$ with untracked predicate), `filterListByTracked` (3 sites, charging $|xs| + 1$ with untracked predicate), `findSomeListByTracked` (1 site, charging $\leq |xs| + 1$ with untracked callback), `tabulateFinsetTracked` (1 site, charging $\sum_{x \in S}(f\ x).2 + 1$), `sumFinsetTracked` (1 site, charging $\sum_{x \in S}(f\ x).2 + 1$), `evalSupportScanTracked` (2 sites, charging $|U||V| + 1$ with untracked predicate), `evalMinNbrScanTracked` (1 site, charging $\sum_{xz \in S}(1 + \min(|N_1\ xz|, |N_2\ xz|)) + 1$), and `scanBlockExtractTracked` (1 site, charging $3m^2 + 1$).
- *Underlying data-representation assumptions*: Tier-B charges model matrices as $O(1)$-access 2D arrays (with copy-on-write or subrectangle-mask views where an older residual matrix is still read), finite vertex sets (`Finset`) as $O(1)$-membership/cardinality/indexing structures, real density/degree thresholds in the specification as $O(\log n)$-bit rational cross-multiplications, and row/column degrees and dyadic buckets as memoized within a single charged scan step.

107-Site Table of Free-Payload `CostM` Combinator Call Sites

The table below is a regex snapshot of `def` bodies produced by the hygiene script `lean/scripts/gen_payload_table.py` (checking unique marker delimitation, exact table block synchronization, and absence of raw `Prod.mk` tuples outside `CostMonadAndLinter.lean`, without full Lean name/type resolution). Of the **107** definition-level call sites, **106** lie on the call graph reachable from the five headline roots (`computableBMMTracked`, `computableTriangleDetectionTracked`, `cappedABDecompositionTracked`, `cappedADecompositionConcreteTracked`, `twoStage_compose`; `aDecompositionConcreteTracked` delegates to `aDecompositionTracked` with `concreteOracleVTracked`, sharing the same `aDecompositionLoopTracked` call graph as `cappedADecompositionConcreteTracked`) and **1** belongs to the off-path standalone small-universe helper `constructSmallTwoStageSamplerTracked` (in row 67, `bucketedRecTracked` at `Decomp3DPotential.lean:176` executes its `scanMatrixTracked Y' Z'` once per wrapper call). Script reachability walks textual identifiers inside `def` bodies (covering structure-field dispatch via the explicit `twoStage_compose` root), whereas the Lean-level `#verify_headline_cone_excludes` traverses kernel `ConstantInfo.defnInfo` expressions:

#	Module:Line & Enclosing `def`	Combinator	On Path	Payload Snippet	Charging Rationale
1	`ObliviousSampler.lean:551 transportFamTracked`	`retTracked`	yes	`(indexFinset Y i)) T_idx) fam_idx`	Unit return (+1, `retTracked_snd`); $O(1)$ array index under Class B.
2	`ObliviousSampler.lean:630 lookupSeqTracked`	`retTracked`	yes	`(T1.getD j.val y0)) I2`	Unit return (+1, `retTracked_snd`); $O(1)$ array index under Class B.
3	`ObliviousSampler.lean:1047 modFinTracked`	`retTracked`	yes	`(if hr : r < M then <r, hr> else <0, hM>))`	Unit return (+1, `retTracked_snd`) packaging already-bound values/accumulators.
4	`ObliviousSampler.lean:1287 stage2FinMFamTracked`	`retTracked`	yes	`[[]]`	Unit return (+1, `retTracked_snd`) packaging already-bound values/accumulators.
5	`ObliviousSampler.lean:1775 twoStageConstructTracked`	`retTracked`	yes	`[[]]`	Unit return (+1, `retTracked_snd`) packaging already-bound values/accumulators.
6	`ObliviousSampler.lean:2519 constructSmallTwoStageSamplerTracked`	`retTracked`	no	`(constructSmallTwoStage Sampler Y invEps logInvDelta)`	Off-headline helper/variant; Unit return (+1, `retTracked_snd`) for `constructSmall TwoStageSampler` (uncharged `[Y.toList]` in off-path helper).
7	`ApproxGridNorm.lean:1279 prodTupleBoolTracked`	`retTracked`	yes	`1 | y :: ys => CostM.bind (prodTupleBoolTracked A x ys) (fun ...`	Unit return (+1, `retTracked_snd`) packaging already-bound values/accumulators.
8	`ApproxGridNorm.lean:1282 prodTupleBoolTracked`	`retTracked`	yes	`(if A x y then p else 0))`	Unit return (+1, `retTracked_snd`) packaging already-bound values/accumulators.
9	`ApproxGridNorm.lean:1388 buildTupleTableFinsetTracked`	`sumFinsetTracked`	yes	`X (fun x' => prodTuple BoolTracked A x' ys)) (fun c =>`	Charges $\sum_{x \in S}(f\ x).2 + 1$ (`sumFinsetTracked_snd`; no domain-size term; each callback charges ≥ 1).

#	Module:Line & Enclosing def	Combinator	On Path	Payload Snippet	Charging Rationale
10	ApproxGridNorm.lean:1389 buildTupleTableFinsetTracked	retTracked	yes	(ys, c))) (piList T_list d)	Unit return (+1, retTracked_snd) packaging already-bound values/accumulators.
11	ApproxGridNorm.lean:1419 contractTupleTableSimpleTracked	retTracked	yes	(if px = 0 then acc else acc + entry.2))) 0 table	Unit step charge (+1, retTracked_snd) for table-contraction accumulator addition ($O(d \log B + \log \lvert X \rvert)$-bit counter; see Class B).
12	ApproxGridNorm.lean:1477 contractTupleTableFamilySimpleTracked	retTracked	yes	(acc + v))) 0 tables	Unit step charge (+1, retTracked_snd) for table-contraction accumulator addition ($O(d \log B + \log \lvert X \rvert)$-bit counter; see Class B).
13	ApproxGridNorm.lean:1503 approxGridNormVxRowPairTracked	filterFinsetByTracked	yes	Y (fun y => A x y)) (fun nbrs =>	Charges $\lvert S \rvert + 1$ (filterFinsetByTracked_snd); predicate is an untracked Bool function ($O(1)$-word work under Class B).
14	ApproxGridNorm.lean:1509 approxGridNormVxRowPairTracked	retTracked	yes	(num, den)))))))	Unit return (+1, retTracked_snd) packaging already-bound values/accumulators.
15	MinDegreeAndDyadicSift.lean:212 minDegreeLoopTracked	scanFinsetTracked	yes	U' (U'.erase (choose MinBadRow U' V M eps0 hbad), 2)	Charges $\lvert U \rvert + 1$ (scanFinsetTracked_snd).
16	MinDegreeAndDyadicSift.lean:214 minDegreeLoopTracked	scanFinsetTracked	yes	U' ()) (fun _ => minDegree LoopTracked U V M eps0 gamma0 (U'.erase (...	Charges $\lvert U \rvert + 1$ (scanFinsetTracked_snd).
17	MinDegreeAndDyadicSift.lean:217 minDegreeLoopTracked	scanFinsetTracked	yes	U' (U', 1) termination_by U'.card decreasing_by have hu : cho...	Charges $\lvert U \rvert + 1$ (scanFinsetTracked_snd).
18	MinDegreeAndDyadicSift.lean:234 minDegreeTracked	scanMatrixTracked	yes	U V ()) (fun _ => minDegreeLoopTracked U V M eps0 gamma0 U)	Charges $\lvert U \rvert \lvert V \rvert + 1$ (scanMatrixTracked_snd).
19	MinDegreeAndDyadicSift.lean:1023 dyadicSiftTracked	scanMatrixTracked	yes	X Y (if (eps / 16) * (((2 : R) ^ ((s + 2) * d))^(-1)) * (X.card : ...	Charges $\lvert U \rvert \lvert V \rvert + 1$ (scanMatrixTracked_snd).
20	MinDegreeAndDyadicSift.lean:1033 dyadicSiftTracked	retTracked	yes	("subrect", outerDyadic Bucket X Y A delta s, Y, s)	Unit return (+1, retTracked_snd); submatrix/bucket scan is charged by the paired scan*Tracked call in the same step.
21	MinDegreeAndDyadicSift.lean:1036 dyadicSiftTracked	scanFinsetTracked	yes	X (selectWitnessRow X Y A delta d V hwit)) (fun x_star =>	Charges $\lvert U \rvert + 1$ (scanFinsetTracked_snd).
22	MinDegreeAndDyadicSift.lean:1041 dyadicSiftTracked	scanMatrixTracked	yes	X Y (if (eps / 16) * (((2 : R) ^ ((s + 2) * d))^(-1)) * (X.card : ...	Charges $\lvert U \rvert \lvert V \rvert + 1$ (scanMatrixTracked_snd).
23	MinDegreeAndDyadicSift.lean:1051 dyadicSiftTracked	retTracked	yes	("subrect", innerDyadic Bucket X Y' A delta s, Y', s)	Unit return (+1, retTracked_snd); submatrix/bucket scan is charged by the paired scan*Tracked call in the same step.
24	MinDegreeAndDyadicSift.lean:1053 dyadicSiftTracked	retTracked	yes	("regular", X, Y, 0))) else retTracked ("regular", X, Y, 0)))	Unit return (+1, retTracked_snd) packaging already-bound values/accumulators.
25	MinDegreeAndDyadicSift.lean:1055 dyadicSiftTracked	retTracked	yes	("regular", X, Y, 0)))	Unit return (+1, retTracked_snd) packaging already-bound values/accumulators.
26	MinDegreeAndDyadicSift.lean:2040 concreteApproxGridNormVTracked	tabulateFinsetTracked	yes	X (fun x => CostM.bind (approx GridNormVxRowPairTracked X Y ta...	Charges $\sum_{x \in S} (f\ x).2 + 1$ (tabulateFinsetTracked_snd; no domain-size term; each callback charges ≥ 1).
27	MinDegreeAndDyadicSift.lean:2043 concreteApproxGridNormVTracked	retTracked	yes	(ratPairToReal q)))))))	Unit return (+1, retTracked_snd) packaging already-bound values/accumulators.
28	Decomp2D.lean:271 goodRectStepTracked	scanMatrixTracked	yes	X' Y' (Sum.inr (X', Y', RectTag.aBoosted, s_bar))	Charges $\lvert U \rvert \lvert V \rvert + 1$ (scanMatrixTracked_snd).
29	Decomp2D.lean:279 goodRectStepTracked	retTracked	yes	(Sum.inl (X1, Y', s_bar, 0))	Unit return (+1, retTracked_snd) packaging already-bound values/accumulators.
30	Decomp2D.lean:281 goodRectStepTracked	retTracked	yes	(Sum.inr (X1, Y', RectTag.aBoosted, s_bar))	Unit return (+1, retTracked_snd) packaging already-bound values/accumulators.

#	Module:Line & Enclosing def	Combinator	On Path	Payload Snippet	Charging Rationale
31	Decomp2D.lean:287 goodRectStepTracked	retTracked	yes	(Sum.inr (X1, Y', RectTag.regular, s_bar))	Unit return (+1, retTracked_snd) packaging already-bound values/accumulators.
32	Decomp2D.lean:290 goodRectStepTracked	retTracked	yes	(Sum.inl (resSift.2.1, resSift.2.2.1, s_bar + s, s))))	Unit return (+1, retTracked_snd) packaging already-bound values/accumulators.
33	Decomp2D.lean:337 goodRectLoopTracked	retTracked	yes	(X', Y', RectTag.aBoosted, s_bar, steps_done)	Unit return (+1, retTracked_snd) packaging already-bound values/accumulators.
34	Decomp2D.lean:342 goodRectLoopTracked	retTracked	yes	(X_out, Y_out, tag_out, s_final, steps_done)	Unit return (+1, retTracked_snd) packaging already-bound values/accumulators.
35	Decomp2D.lean:418 aDecompositionLoopTracked	scanMatrixTracked	yes	X Y (acc.reverse ++ [<X, Y, restrictSubrect A_rem X Y, RectTa...	Charges $\|U\|\|V\|+1$ (scanMatrixTracked_snd).
36	Decomp2D.lean:422 aDecompositionLoopTracked	scanMatrixTracked	yes	X Y (acc.reverse ++ [<X, Y, restrictSubrect A_rem X Y, RectTa...	Charges $\|U\|\|V\|+1$ (scanMatrixTracked_snd).
37	Decomp2D.lean:431 aDecompositionLoopTracked	scanMatrixTracked	yes	X Y ()) (fun _ => aDecompositionLoopTracked fuel maxIter X Y ...	Charges $\|U\|\|V\|+1$ (scanMatrixTracked_snd).
38	DyadicBucketingAndGoodCube.lean:203 dyadicColBucketTracked	scanMatrixTracked	yes	X' Y' (dyadicColBucket X' Y' A' mu j)	Charges $\|U\|\|V\|+1$ (scanMatrixTracked_snd).
39	DyadicBucketingAndGoodCube.lean:219 dyadicColBucketingLoopTracked	retTracked	yes	[] | j :: js => CostM.bind (dyadic ColBucketTracked X' Y' A' mu...	Unit return (+1, retTracked_snd) packaging already-bound values/accumulators.
40	DyadicBucketingAndGoodCube.lean:223 dyadicColBucketingLoopTracked	retTracked	yes	((j, b) :: rest)))	Unit return (+1, retTracked_snd) packaging already-bound values/accumulators.
41	DyadicBucketingAndGoodCube.lean:944 cappedGoodRect3DLoopTracked	scanMatrixTracked	yes	X Y { X_ell := X, Y_ell := Y, A_ell := submatZeroExt X Y A,	Charges $\|U\|\|V\|+1$ (scanMatrixTracked_snd).
42	DyadicBucketingAndGoodCube.lean:949 cappedGoodRect3DLoopTracked	scanMatrixTracked	yes	X Y { X_ell := X, Y_ell := Y, A_ell := submatZeroExt X Y A,	Charges $\|U\|\|V\|+1$ (scanMatrixTracked_snd).
43	DyadicBucketingAndGoodCube.lean:958 cappedGoodRect3DLoopTracked	scanMatrixTracked	yes	X Y { sub with steps := sub.steps + 1 })	Charges $\|U\|\|V\|+1$ (scanMatrixTracked_snd).
44	DyadicBucketingAndGoodCube.lean:960 cappedGoodRect3DLoopTracked	scanMatrixTracked	yes	X Y { X_ell := X', Y_ell := Y, A_ell := submat ZeroExt X' Y A,	Charges $\|U\|\|V\|+1$ (scanMatrixTracked_snd).
45	DyadicBucketingAndGoodCube.lean:970 cappedGoodRect3DLoopTracked	scanMatrixTracked	yes	X Y { sub with S_ell := sub.S_ell + s, steps := sub.steps + 1 })	Charges $\|U\|\|V\|+1$ (scanMatrixTracked_snd).
46	DyadicBucketingAndGoodCube.lean:973 cappedGoodRect3DLoopTracked	scanMatrixTracked	yes	X Y { X_ell := X', Y_ell := Y, A_ell := submat ZeroExt X' Y A,	Charges $\|U\|\|V\|+1$ (scanMatrixTracked_snd).
47	DyadicBucketingAndGoodCube.lean:1640 cappedADecomp3DLoopTracked	scanMatrixTracked	yes	X Y [<X, Y, submatZeroExt X Y A_rem, RectTag.residual, 0>]	Charges $\|U\|\|V\|+1$ (scanMatrixTracked_snd).
48	DyadicBucketingAndGoodCube.lean:1643 cappedADecomp3DLoopTracked	scanMatrixTracked	yes	X Y [<X, Y, submatZeroExt X Y A_rem, RectTag.residual, 0>]	Charges $\|U\|\|V\|+1$ (scanMatrixTracked_snd).
49	DyadicBucketingAndGoodCube.lean:1649 cappedADecomp3DLoopTracked	scanMatrixTracked	yes	X Y (piece :: rest)))	Charges $\|U\|\|V\|+1$ (scanMatrixTracked_snd).
50	DyadicBucketingAndGoodCube.lean:2165 processGoodCubePieceTracked	scanMatrixTracked	yes	P.X_sub P.Y_sub (Sum.inr <P.X_sub, P.Y_sub, Z_L, P.A_sub, Rec...	Charges $\|U\|\|V\|+1$ (scanMatrixTracked_snd).
51	DyadicBucketingAndGoodCube.lean:2169 processGoodCubePieceTracked	scanMatrixTracked	yes	Y' Z_curr (Sum.inl (P.Y_sub, Z_curr, 0))	Charges $\|U\|\|V\|+1$ (scanMatrixTracked_snd).
52	DyadicBucketingAndGoodCube.lean:2171 processGoodCubePieceTracked	scanMatrixTracked	yes	Y' Z_curr (Sum.inr <P.X_sub, P.Y_sub, empty, P.A_sub, RectTag.aBo...	Charges $\|U\|\|V\|+1$ (scanMatrixTracked_snd).
53	DyadicBucketingAndGoodCube.lean:2175 processGoodCubePieceTracked	scanMatrixTracked	yes	Y' Z_curr (Sum.inl (P.Y_sub, Z_curr, 0))	Charges $\|U\|\|V\|+1$ (scanMatrixTracked_snd).
54	DyadicBucketingAndGoodCube.lean:2180 processGoodCubePieceTracked	scanMatrixTracked	yes	Y' Z_curr (Sum.inl (P.Y_sub, Z_ell, 0))	Charges $\|U\|\|V\|+1$ (scanMatrixTracked_snd).
55	DyadicBucketingAndGoodCube.lean:2184 processGoodCubePieceTracked	scanMatrixTracked	yes	Y' Z_curr (Sum.inl (siftZ.2.2.1, siftZ.2.1, siftZ.2.2.2))	Charges $\|U\|\|V\|+1$ (scanMatrixTracked_snd).
56	DyadicBucketingAndGoodCube.lean:2186 processGoodCubePieceTracked	scanMatrixTracked	yes	Y' Z_curr (Sum.inr <P.X_sub, P.Y_sub, Z_ell, P.A_sub, RectTag...	Charges $\|U\|\|V\|+1$ (scanMatrixTracked_snd).
57	DyadicBucketingAndGoodCube.lean:2239 processGoodCubePiecesTracked	retTracked	yes	(Sum.inr []) | P :: rest => CostM.bind (process GoodCubePieceT...	Unit return (+1, retTracked_snd) packaging already-bound values/accumulators.

#	Module:Line & Enclosing def	Combinator	On Path	Payload Snippet	Charging Rationale
58	DyadicBucketingAndGoodCube.lean:2243 processGoodCubePiecesTracked	retTracked	yes	(Sum.inl step) | Sum.inr sl => CostM.bind (process GoodCubePie...	Unit return (+1, retTracked_snd) packaging already-bound values/accumulators.
59	DyadicBucketingAndGoodCube.lean:2247 processGoodCubePiecesTracked	retTracked	yes	(Sum.inl step) | Sum.inr sls => retTracked (Sum.inr (sl :: sl...	Unit return (+1, retTracked_snd) packaging already-bound values/accumulators.
60	DyadicBucketingAndGoodCube.lean:2248 processGoodCubePiecesTracked	retTracked	yes	(Sum.inr (sl :: sls))))	Unit return (+1, retTracked_snd) packaging already-bound values/accumulators.
61	DyadicBucketingAndGoodCube.lean:2331 cappedGoodCubeLoopTracked	scanMatrixPairTracked	yes	X Y_curr Z_curr { mode := CubeMode.bBoost X_star := X	Charges $\|X\|\|Y\| + \|Y\|\|Z\| + 1$ (scanMatrixPairTracked_snd).
62	DyadicBucketingAndGoodCube.lean:2345 cappedGoodCubeLoopTracked	scanMatrixPairTracked	yes	X Y_curr Z_curr { mode := CubeMode.bBoost X_star := X	Charges $\|X\|\|Y\| + \|Y\|\|Z\| + 1$ (scanMatrixPairTracked_snd).
63	DyadicBucketingAndGoodCube.lean:2363 cappedGoodCubeLoopTracked	scanMatrixPairTracked	yes	X Y_curr Z_curr { sub with steps := sub.steps + 1 })	Charges $\|X\|\|Y\| + \|Y\|\|Z\| + 1$ (scanMatrixPairTracked_snd).
64	DyadicBucketingAndGoodCube.lean:2374 cappedGoodCubeLoopTracked	scanMatrixPairTracked	yes	X Y_curr Z_curr { sub with S_B := sub.S_B + s_step, steps := ...	Charges $\|X\|\|Y\| + \|Y\|\|Z\| + 1$ (scanMatrixPairTracked_snd).
65	DyadicBucketingAndGoodCube.lean:2377 cappedGoodCubeLoopTracked	scanMatrixPairTracked	yes	X Y_curr Z_curr { mode := CubeMode.normal X_star := X	Charges $\|X\|\|Y\| + \|Y\|\|Z\| + 1$ (scanMatrixPairTracked_snd).
66	Decomp3DPotential.lean:166 bucketedRecTracked	filterListByTracked	yes	(fun b : N x Finset beta => decide b.2.Nonempty) buckets)	Charges $\|xs\| + 1$ (filterListByTracked_snd); predicate untracked.
67	Decomp3DPotential.lean:176 bucketedRecTracked	scanMatrixTracked	yes	Y' Z' res)))	Charges $\|U\|\|V\| + 1$ (scanMatrixTracked_snd).
68	Decomp3DPotential.lean:861 baseCase2DecompositionTracked	scanMatrixPairTracked	yes	X Y Z [sparseCubePiece X Y Z A B]	Charges $\|X\|\|Y\| + \|Y\|\|Z\| + 1$ (scanMatrixPairTracked_snd).
69	Decomp3DPotential.lean:865 baseCase2DecompositionTracked	scanMatrixPairTracked	yes	X Y Z (chunkCubePiece X Y Z A B chunk))	Charges $\|X\|\|Y\| + \|Y\|\|Z\| + 1$ (scanMatrixPairTracked_snd).
70	Decomp3DPotential.lean:1962 rawDeltaHBNatTracked	retTracked	yes	(max 1 s))))))))	Unit return (+1, retTracked_snd) packaging already-bound values/accumulators.
71	Decomp3DPotential.lean:1980 rawDeltaHANatTracked	retTracked	yes	(max 1 s))))))))	Unit return (+1, retTracked_snd) packaging already-bound values/accumulators.
72	Decomp3DPotential.lean:2373 clampDeltaHBTracked	retTracked	yes	(min remDepth rawNat))	Unit return (+1, retTracked_snd) packaging already-bound values/accumulators.
73	Decomp3DPotential.lean:2391 clampDeltaHATracked	retTracked	yes	(min remDepth rawNat))	Unit return (+1, retTracked_snd) packaging already-bound values/accumulators.
74	Decomp3D.lean:272 sStarLoopTracked	scanMatrixPairTracked	yes	X Y Z [baseCase1Piece X Y Z A B eps D]	Charges $\|X\|\|Y\| + \|Y\|\|Z\| + 1$ (scanMatrixPairTracked_snd).
75	Decomp3D.lean:274 sStarLoopTracked	scanMatrixPairTracked	yes	X Y Z (isSparse3DPiece X Y Z A B D)) (fun isSparse =>	Charges $\|X\|\|Y\| + \|Y\|\|Z\| + 1$ (scanMatrixPairTracked_snd).
76	Decomp3D.lean:276 sStarLoopTracked	scanMatrixPairTracked	yes	X Y Z [baseCase1Piece X Y Z A B eps D]	Charges $\|X\|\|Y\| + \|Y\|\|Z\| + 1$ (scanMatrixPairTracked_snd).
77	Decomp3D.lean:287 sStarLoopTracked	scanMatrixTracked	yes	cubeRes.Y_star cubeRes.Z_star	Charges $\|U\|\|V\| + 1$ (scanMatrixTracked_snd).
78	Decomp3D.lean:291 sStarLoopTracked	scanMatrixTracked	yes	Y Z (supportCountRealNat Y Z B))	Charges $\|U\|\|V\| + 1$ (scanMatrixTracked_snd).
79	Decomp3D.lean:306 sStarLoopTracked	scanMatrixPairTracked	yes	X Y Z S_out))))) else CostM.bind (filter ListByTracked (fun sl...	Charges $\|X\|\|Y\| + \|Y\|\|Z\| + 1$ (scanMatrixPairTracked_snd).
80	Decomp3D.lean:309 sStarLoopTracked	filterListByTracked	yes	(fun sl => decide sl.Z_ell.Nonempty) cube Res.slices)	Charges $\|xs\| + 1$ (filterListByTracked_snd); predicate untracked.
81	Decomp3D.lean:313 sStarLoopTracked	scanMatrixTracked	yes	sl.Y_ell sl.Z_ell (direct SlicePiece B D sl))	Charges $\|U\|\|V\| + 1$ (scanMatrixTracked_snd).
82	Decomp3D.lean:317 sStarLoopTracked	filterListByTracked	yes	(fun sl => decide (cube Res.Z_star \sl.Z_ell).Nonempty)	Charges $\|xs\| + 1$ (filterListByTracked_snd); predicate untracked.
83	Decomp3D.lean:328 sStarLoopTracked	scanMatrixTracked	yes	sl.X_ell sl.Y_ell (support CountRealNat sl.X_ell sl.Y_ell sl.A...	Charges $\|U\|\|V\| + 1$ (scanMatrixTracked_snd).

#	Module:Line & Enclosing def	Combinator	On Path	Payload Snippet	Charging Rationale
84	Decomp3D.lean:343 sStarLoopTracked	scanMatrixPairTracked	yes	X Y Z S_out)))))))))	Charges $\|X\|\|Y\| + \|Y\|\|Z\| + 1$ (scanMatrixPairTracked_snd).
85	Decomp3D.lean:429 cappedABDecompAuxTracked	scanMatrixPairTracked	yes	X Y Z (isSparse3DPiece X Y Z A B D)) (fun isSparse =>	Charges $\|X\|\|Y\| + \|Y\|\|Z\| + 1$ (scanMatrixPairTracked_snd).
86	Decomp3D.lean:431 cappedABDecompAuxTracked	scanMatrixPairTracked	yes	X Y Z [baseCase1Piece X Y Z A B eps D]	Charges $\|X\|\|Y\| + \|Y\|\|Z\| + 1$ (scanMatrixPairTracked_snd).
87	TriangleAndBMM.lean:436 checkSparsePieceNatTracked	retTracked	yes	(cA || cB))))))	Unit return (+1, retTracked_snd) packaging already-bound values/accumulators.
88	TriangleAndBMM.lean:534 detectOnPieceNatTracked	scanMatrixPairTracked	yes	P.X_k P.Y_k P.Z_k ()) (fun _ =>	Charges $\|X\|\|Y\| + \|Y\|\|Z\| + 1$ (scanMatrixPairTracked_snd).
89	TriangleAndBMM.lean:548 detectOnPieceNatTracked	evalSupportScanTracked	yes	suppB P.X_k (fun yz x => decide (0 < P.A_k x yz.1 and C x yz.2 ...	Charges $\|U\|\|V\| + 1$ (evalSupportScanTracked_snd); predicate untracked.
90	TriangleAndBMM.lean:551 detectOnPieceNatTracked	evalSupportScanTracked	yes	suppA P.Z_k (fun xy z => decide (0 < P.B_k xy.2 z and C xy.1 z ...	Charges $\|U\|\|V\| + 1$ (evalSupportScanTracked_snd); predicate untracked.
91	TriangleAndBMM.lean:554 detectOnPieceNatTracked	scanMatrixTracked	yes	P.X_k P.Z_k ()) (fun _ => let S_k := (P.X_k x P.Z_k).filter ...	Charges $\|U\|\|V\| + 1$ (scanMatrixTracked_snd).
92	TriangleAndBMM.lean:560 detectOnPieceNatTracked	retTracked	yes	true else let N_A : Fin n -> Finset (Fin n) := fun x => P.Y_k....	Unit return (+1, retTracked_snd) packaging already-bound values/accumulators.
93	TriangleAndBMM.lean:566 detectOnPieceNatTracked	evalMinNbrScanTracked	yes	S_k (fun xz => N_A xz.1) (fun xz => N_B xz.2) (fun xz y => de...	Charges $\sum_{xz \in S}(1 + \min(\|N_1\ xz\|, \|N_2\ xz\|)) + 1$ (evalMinNbrScanTracked_snd).
94	TriangleAndBMM.lean:725 triangleDetectionOnPiecesTracked	retTracked	yes	(acc || r))) false S	Unit return (+1, retTracked_snd) packaging already-bound values/accumulators.
95	TriangleAndBMM.lean:2422 triDetParamDTracked	retTracked	yes	q else retTracked 320))))	Unit return (+1, retTracked_snd) packaging already-bound values/accumulators.
96	TriangleAndBMM.lean:2422 triDetParamDTracked	retTracked	yes	320))))	Unit return (+1, retTracked_snd) packaging already-bound values/accumulators.
97	TriangleAndBMM.lean:2506 computableTriangleDetectionTracked_of_sampler	retTracked	yes	ans))))	Unit return (+1, retTracked_snd) packaging already-bound values/accumulators.
98	TriangleAndBMM.lean:2963 blockSizeBMMTracked	retTracked	yes	s2 else retTracked 1)))	Unit return (+1, retTracked_snd) packaging already-bound values/accumulators.
99	TriangleAndBMM.lean:2963 blockSizeBMMTracked	retTracked	yes	1)))	Unit return (+1, retTracked_snd) packaging already-bound values/accumulators.
100	TriangleAndBMM.lean:3136 findBlockTriangleTracked	findSomeListByTracked	yes	(checkBlockSubTriple n m a b c A B C) (block SubTriplesList m)	Charges $\le \|xs\| + 1$ (findSomeListByTracked_snd_le; $\le m^3 + 1$ bound used); callback untracked; m^3-list blockSubTriplesList m uncharged.
101	TriangleAndBMM.lean:3271 processBlockTripleLoopTracked	retTracked	yes	C | fuel + 1, C => let A_ab := extractBlockMat n m a b A	Unit return (+1, retTracked_snd) packaging already-bound values/accumulators.
102	TriangleAndBMM.lean:3276 processBlockTripleLoopTracked	scanBlockExtractTracked	yes	m ()) (fun _ => CostM.bind (triDetTracked A_ab B_bc C_ac) (fu...	Charges $3m^2 + 1$ (scanBlockExtractTracked_snd).
103	TriangleAndBMM.lean:3282 processBlockTripleLoopTracked	retTracked	yes	(clearMatEntry C xz.1 xz.2)) (fun C_next =>	Unit return (+1, retTracked_snd) for single-cell update clearMatEntry ($O(1)$ 2D-array write under Class B).
104	TriangleAndBMM.lean:3285 processBlockTripleLoopTracked	retTracked	yes	C) else retTracked C))	Unit return (+1, retTracked_snd) packaging already-bound values/accumulators.

#	Module:Line & Enclosing def	Combinator	On Path	Payload Snippet	Charging Rationale
105	TriangleAndBMM.lean:3287 processBlockTripleLoopTracked	retTracked	yes	C))	Unit return (+1, retTracked_snd) packaging already-bound values/accumulators.
106	TriangleAndBMM.lean:3625 bmmFromTriDetTracked	scanMatrixTracked	yes	U U (fun _ _ : Fin n => true)) (fun initC =>	Charges $\|U\|\|V\|+1$ (scanMatrixTracked_snd).
107	TriangleAndBMM.lean:3630 bmmFromTriDetTracked	scanMatrixTracked	yes	U U (fun x z => !finalC x z)))	Charges $\|U\|\|V\|+1$ (scanMatrixTracked_snd).

*Additional 12 statement/proof/docstring occurrences of scan*Tracked (making 63 total scan*Tracked grep matches outside CostMonadAndLinter.lean)*: MinDegreeAndDyadicSift.lean:1012 (docstring), MinDegreeAndDyadicSift.lean:1088 (proof), MinDegreeAndDyadicSift.lean:1097 (proof), MinDegreeAndDyadicSift.lean:1122 (proof), MinDegreeAndDyadicSift.lean:1131 (proof), Decomp3DPotential.lean:157 (docstring), Decomp3DPotential.lean:855 (docstring), Decomp3D.lean:745 (proof), Decomp3D.lean:751 (proof), Decomp3D.lean:771 (proof), TriangleAndBMM.lean:1176 (statement), TriangleAndBMM.lean:1191 (statement).

Compile-Time AST Linter, Dead-Code Linter & RAM Model Scope:

- *Compile-Time Hygiene Lints (#verify_tracked_declarations_ast_audit, #verify_no_unused_decls & #verify_headline_cone_excludes, CostMonadAndLinter.lean)*: All five metaprogramming commands in CostMonadAndLinter.lean are **heuristic hygiene lints for defense in depth, not a soundness guard**: #verify_tracked_declarations_ast_audit (invoked at the top-level package root BMMGrid.lean) checks that every entry in forbiddenFormulaNamesInTracked, forbiddenOracleNamesInHelpers, and computableComplexityConstants exists in the environment and checks every *Tracked definition and every non-auxiliary BMMGrid.* definition returning CostM, flagging references to closed-form complexity formulas (forbiddenFormulaNamesInTracked), discarded non-Unit CostM.bind results (verifyNoDiscardedBind), direct Prod.mk tuple returns in non-primitive CostM tail positions (verifyNoRawCostMTuple), or transitive calls to denylisted untracked moment/composition/norm helpers over ConstantInfo.defnInfo (forbiddenOracleNamesInHelpers). In addition, #verify_headline_cone_excludes runs an unbounded-depth BFS over ConstantInfo.defnInfo from the five headline roots (computableBMMTracked, computableTriangleDetectionTracked, cappedABDecompositionTracked, cappedADecompositionConcreteTracked, twoStage_compose, reaching 228 BMMGrid.* definitions) to check that no specification function (boolMatMul, hasTriangle, stdMatMul, gridNorm*, Stage1Sampler.fam, Stage2Sampler.fam, ...) is referenced by any definition reachable from the headline algorithms, while #verify_no_unused_decls checks that every BMMGrid.* declaration not exempted by isCompilerOrInductAux (instances, structure projections, inductive eliminators, and compiler/derived-type helpers) is reachable from headlineTheoremRoots, auxiliaryRoots, or linterHelperRoots. Because CostM α is an abbrev for $\alpha \times \mathbb{N}$, manual inspection of the 107 combinator call sites above remains the basis for cost-model review.
- *Relation to Word-RAM / Cook–Reckhow Log-Cost RAM (§1.2 Theorem 1.2 & Appendix C \label{app:models})*: The Lean formalization establishes the **unit-cost Arithmetic / Word-RAM** declared-charge bound (CostM step count t in thm_main_triangle_detection, thm_main_triangle_detection_graph, and thm_main_bmm) and the **Cook–Reckhow log-cost RAM** absorption bound $(t \cdot 12(\lfloor \log_2 n \rfloor + 5)^2 = t \cdot 12(\texttt{bitLengthNat}\ n + 4)^2$ in thm_main_triangle_detection_log_cost_absorption and thm_main_bmm_log_cost_absorption, with aliases thm_main_triangle_detection_log_cost and thm_main_bmm_log_cost). The correspondence between CostM declared charges and the Appendix C machine models (including the pointer-machine model) is argued in the paper and is not machine-checked in Lean. A verified refinement of the algorithm to the RAM model is outside the scope of this artifact and is not planned.
- *Scope of BMMGrid/ArithRAM.lean*: ArithRAM.lean formalizes a register-addressed RAM (ArithOp constructors loadImm, copy, loadInd, storeInd, add, sub, mul, div, mod and ArithStmt constructors op, branchLt, branchEq, halt) and proves execution bounds for **four standalone 2-to-6-instruction micro-kernels** (ramInitAndAdd, ramReadMatrixEntry, ramWriteMatrixEntry, ramCheckTriangleTriple) plus one $(2n+3)$-step counter loop (ramCountLoop). The top-level functions computableTriangleDetectionTracked and computableBMMTracked are **not** compiled to an ArithRAM; thm_main_triangle_detection_log_cost_absorption and thm_main_bmm_log_cost_absorption (TriangleAndBMM.lean, aliased as *_log_cost) are conjunctions of those standalone micro-kernel witnesses with the CostM step bound t multiplied by pointerBitOverhead $n =$ bitLengthNat $n + 4$ and $12 \cdot (\texttt{pointerBitOverhead}\ n)^2$.

4 Class C — Statement Conventions & Theorem Packaging

1. **Separation of Structural/Accuracy Theorems and `CostM` Time-Bound Theorems**: In several modules, a paper theorem or lemma that states both structural/accuracy guarantees and an algorithmic running-time bound is split in Lean into a structural theorem on the untracked function and a companion `.2` cost theorem on the `*Tracked` function (connected by a `@[simp]` projection theorem `(fooTracked ...).1 = foo ...`):
   - *Theorem 1.1 (`\label{thm:main-decomp}`) & Direct Sampler Corollary (`thm_main_decomp_of_samplers`)*: `thm_main_decomp` (`Decomp3D.lean`) proves the structural, entrywise dominance, piece-count, $C_{1/4,1/4}$ potential, and symbolic per-piece `CostM` step-count conclusions of Theorem 1.1 on `cappedABDecomposition` / `cappedABDecompositionTracked` for all $0 < \varepsilon \leq 1/160$ given `S : TwoStageSamplerOracle` β ε, and `thm_main_decomp_of_samplers` (`Decomp3D.lean`) discharges the oracle directly via `twoStage_compose` `S₁` `S₂` ε `hε₀` from `Stage1Sampler` and `Stage2Sampler` for all $0 < \varepsilon \leq 1/160$. The closed-form algorithmic time bound at $[n]^3$ and $\varepsilon = 1/160$ is proved in `cappedABDecomposition_concrete_bounds` and `thm_main_3d_decomposition` (`TriangleAndBMM.lean`) over `cappedABDecompositionTracked`.
   - *Lemma 3.2 (`\label{lem:min-degree}`), Lemma 4.1 (`\label{lem:approx-grid-norm}`) & Lemma 5.3 (`\label{lem:cube-step}`)*: `lem_min_degree` (`MinDegreeAndDyadicSift.lean`) proves all structural guarantees of `Prune` (including top-level $|U'| \geq \max(1, \lfloor(1-\gamma_0)|U|\rfloor)$ and, in Case 2, exact floor/ceiling cardinalities $|U'| = \lfloor(1-\gamma_0)|U|\rfloor$ and $|U \setminus U'| = \lceil\gamma_0|U|\rceil$ alongside $(1-\gamma_0)|U| - 1 < |U'| \leq (1-\gamma_0)|U|$), while `minDegreeTracked_snd_le` (`MinDegreeAndDyadicSift.lean`) proves its `CostM` cost bound. Similarly, `lem_approx_grid_norm` (`ApproxGridNorm.lean`, for `IsObliviousSampler`) and `lem_approx_grid_norm_seq_real` (`ApproxGridNorm.lean`, for `IsSeqSampler`) prove the accuracy guarantee of `EstimateNorm`, while `concreteApproxGridNormVTracked_snd_le` (`MinDegreeAndDyadicSift.lean`) proves its `CostM` cost bound. In 3D (`DyadicBucketingAndGoodCube.lean`), `cappedGoodCubeLoop_spec` and `thm_3d_decomp_part1` prove both directions of Algorithm 5 Line 5's residual slice assignment (`\label{line:cube-zl}`: $\mathbb{E}[A_\ell] \leq 2^{-D} \Rightarrow Z_\ell = Z^*$ and $\mathbb{E}[A_\ell] > 2^{-D} \Rightarrow Z_\ell = \emptyset$) and both normal-mode density bounds ($\beta^* < 16(1+\varepsilon\gamma/4)\beta_{\text{init}}$ and $\beta^* < 17\beta_{\text{init}}$).
   - *Theorem 4.4, Corollary 4.5 & Lemma 4.6 (`\label{thm:2d-decomp}`)*: In `Decomp2D.lean`, `thm_2d_decomp_concrete` proves an artifact-constant version of Theorem 4.4 for the concrete estimator `concreteOracleV S` (`cappedADecompositionConcrete` / `cappedADecompositionConcreteTracked` at fuel `rDecomp2DFuel` ε_2 D), including its structural partition, zero-extension on $(X \times Y) \setminus (X_\ell \times Y_\ell)$, residual guarantees ($X_{\text{sub}} = X \wedge Y_{\text{sub}} = Y$ and density $\leq \tau_{\text{low}}$), strict $\tau_{\text{low}} < \mathbb{E}[A_\ell]$ and scale $s_\ell \leq D$ for non-residual pieces, column-width lower bound $|Y_\ell| \geq ((2^{2D})^{-1})^{\texttt{rRect2DFuel}\ \varepsilon_2}|Y| = 2^{-168D\lceil 1/\varepsilon_2\rceil}|Y|$ for every non-residual piece (paired with `rRect2DFuel_width_le_paper` proving $2^{-168D\lceil 1/\varepsilon_2\rceil}|Y| \geq 2^{-194D/\varepsilon_2}|Y|$ for $0 < \varepsilon_2 \leq 1/160$), total piece-count bounds (`pieces.length` $\leq$ `rDecomp2DFuel` ε_2 $D + 1$ and non-residual count $\leq 2^D \cdot M_{\text{rect2D}}(\varepsilon_2, D) \cdot 2^{D^2}$), per-scale L_s and L_{reg} counts, Clause 3 mass conservation, area-shrinkage, Riemann-sum telescoping for $p \in (0,1)$ and $p = 1/4$, and `CostM` running-time bound. It is instantiated from `thm_2d_decomp` via `concreteOracleV_isValid` under the density precondition $2^D(1+\varepsilon_2/2)\mathbb{E}[M] > 1$ of Lemma 4.2. The uncapped Corollary 4.5 holds for all $D \geq 2$ in `thm_2d_decomp_uncapped` and `thm_2d_decomp_uncapped_concrete` (`Decomp2D.lean`, instantiating `concreteOracleV S` on `aDecompositionConcrete` / `aDecompositionConcreteTracked`), and `DyadicBucketingAndGoodCube.lean` proves the 3D-context specialization for $0 < \varepsilon \leq 1/160$ (`cappedGoodRect3D_spec`, `cappedADecomp3DLoop_pieces_spec`, `cappedADecomp3D_eq_cappedADecompositionConcrete`).
2. **Two 2D Decomposition Implementations (`Decomp2D.lean` vs. `DyadicBucketingAndGoodCube.lean`)**: `Decomp2D.lean` formalizes the standalone Section 4.3 algorithms (`goodRectLoop`, `cappedGoodRect`, `aDecompositionLoop`, `aDecomposition`, `cappedADecomposition`) parameterized by an abstract oracle `oracleV` satisfying `IsValidOracleV`, together with the concrete instantiations `cappedGoodRectConcrete`, `cappedADecompositionConcrete`, `aDecompositionConcrete`, `thm_2d_decomp_concrete`, and `thm_2d_decomp_uncapped_concrete`. `DyadicBucketingAndGoodCube.lean` defines a self-contained 2D implementation (`cappedGoodRect3D`, `cappedADecomp3D`, proved equal to the concrete `Decomp2D` functions by `cappedGoodRect3D_eq_cappedGoodRectConcrete` and `cappedADecomp3D_eq_cappedADecompositionConcrete`) that is invoked inside `cappedGoodCube` and the 3D decomposition / BMM pipeline.
3. **Tripartite vs. Simple-Graph Triangle Detection (§1.2 Theorem 1.2 `\label{thm:main-bmm}`)** :

`hasTriangle n A B C` (`Defs.lean`) is the tripartite predicate $\exists i, j, k \in \mathrm{Fin}\, n,\ A(i,j) \wedge B(j,k) \wedge C(i,k)$. `thm_main_triangle_detection_graph` specializes $A = B = C =$ `Adj`, and `hasTriangle_eq_true_iff_strictMono_of_symm_irrefl` (`TriangleAndBMM.lean`) proves that for any symmetric, irreflexive adjacency matrix `Adj`, `hasTriangle n Adj Adj Adj = true` $\iff \exists i < j < k$, `Adj i j = true` $\wedge$ `Adj j k = true` $\wedge$ `Adj i k = true`.

4. **Real-Matrix ($\mathbb{R}$) / `Finset` Interface, Uniform `EstimateNorm` Parameterization & `noncomputable` Headline Functions**:
   - *Noncomputability under $\mathbb{R}$ and `Finset`*: In Sections 3–5, matrices and densities are typed over $\mathbb{R}$ (`A : α → β → ℝ`, with Boolean matrices embedded via `bmatToReal`) and vertex subsets are represented as `Finset α` (whose `.toList` enumeration uses `Classical.choice`). Consequently, **all** headline `*Tracked` functions (`computableBMMTracked`, `computableTriangleDetectionTracked`, `cappedABDecompositionTracked`, `concreteApproxGridNormVTracked`, `twoStageConstructTracked` — despite the historical `computable*` prefix) are `noncomputable`: the `CostM` charge t is a mathematical function of the input, not an `#eval`-able executable.
   - *General ε vs. Pipeline Specialization $\varepsilon = 1/160$*: §1.2 Theorem 1.1 (`\label{thm:main-decomp}`) is proved for all $0 < \varepsilon \le 1/160$ and arbitrary finite sets in `thm_main_decomp` and `thm_main_decomp_of_samplers` (`Decomp3D.lean`), whereas the pipeline theorems (`thm_main_3d_decomposition`, `thm_main_triangle_detection`, `thm_main_triangle_detection_graph`, `thm_main_bmm` in `TriangleAndBMM.lean`) specialize to $\varepsilon = 1/160$.
   - *Uniform `EstimateNorm` Accuracy Parameterization (`accExp`)*: Define `accExp` $\varepsilon\ d :=$ `3d + 6 + Nat.clog 2` $\lceil 1/\varepsilon \rceil_+$ (`ObliviousSampler.lean`), which satisfies `accExp` $(1/160)\ d = 3d + 14$ (`accExp_one_div_160`), $3d + 6 \le$ `accExp` $\varepsilon\ d$ (`three_mul_add_six_le_accExp`), and $3d + 14 \le$ `accExp` $\varepsilon\ d$ for $0 < \varepsilon \le 1/160$ (`three_mul_add_fourteen_le_accExp`). In `TwoStageSamplerOracle` / `concreteApproxGridNormV` (`MinDegreeAndDyadicSift.lean`), §4.1 Algorithm 2 (`EstimateNorm`, `\label{alg:estimate-norm}`) is invoked with constructor arguments `invEps` $= 2^{\texttt{accExp}\ \varepsilon\ d}$ and `logInvDelta` $= d \cdot$ `accExp` $\varepsilon\ d$, which by `TwoStageSamplerOracle.is_sampler` and `approxGridNorm_sampler_inst` (`ObliviousSampler.lean`) yields an $(\varepsilon_{\text{samp}}, \delta_{\text{samp}})$-sampler for every $\varepsilon_{\text{samp}} \ge \varepsilon/2^{3d+6}$ and $\delta_{\text{samp}} \ge (\varepsilon/2^{3d+6})^d$ for every $\varepsilon > 0$. Matching Algorithm 2 Line 1 ($\varepsilon' = \alpha'/2$, $\delta' = (\alpha'/2)^d$), Lean's effective accuracy guarantee is $\alpha' = \varepsilon \cdot 2^{-(3d+5)} = (\varepsilon/32) \cdot 2^{-3d}$ — the paper's own worst-case bound over $\delta > 2^{-d}$ ($\alpha' = \varepsilon\delta^3/32 \ge (\varepsilon/32)2^{-3d}$ in the proof of Lemma 4.2).

5. **Empty-Set Expectation Totalization ($0/0 = 0$)**: Division by zero in Lean 4 / Mathlib evaluates to 0 ($a/0 = 0$), so normalized sums and expectations (`expectSet`, `matDensity`, `rowDeg`, `colDeg`, `normMatMul`, `gridNormPow`) are total functions on all `Finset` arguments and evaluate to 0 when a denominator is empty (matching §4.2 Lemma 4.2 for $\deg_A(x) = 0$ and §5.1 Lemma 5.1 for empty buckets $\beta'_j := 0$). All active shrinkage steps explicitly prove non-emptiness ($|X'| \ge 1, |Y'| \ge 1, |Z'| \ge 1$).

6. **Structural Recursion Fuel Counters (`Nat.ceil`) vs. Analytical Step Bounds (§4.3 Lemma 4.3 `\label{lem:find-rect}` & §5.2 Lemma 5.3 `\label{lem:cube-step}`)** : To define total recursive functions in Lean 4 without `partial def`, each `while`-loop takes an explicit `Nat` fuel parameter:
   - *2D `FindRect` Loop*: Lemma 4.3 (`\label{lem:find-rect}`) states the general-W step bound $r \le \min(2D/\varepsilon,\ 1 + 2\log_2 \max(1, W)/\varepsilon)$. Lean formalizes the capped instance ($W \le 2^{48}$) via `rRectFuel` $\varepsilon =$ `rRect2DFuel` $\varepsilon = 84 \cdot$ `Nat.ceil` $(1/\varepsilon)$ (`rRect2DFuel_le_div` proves (`rRect2DFuel` $\varepsilon : \mathbb{R}) \le 97/\varepsilon$ for $0 < \varepsilon \le 1/160$, `pow_rRect2DFuel_ge` proves $(2 : \mathbb{R})^{48} \le (1 + \varepsilon/2)^{\texttt{rRect2DFuel}\ \varepsilon}$, and `rRect2DFuel_width_le_paper` proves $2^{-168D\lceil 1/\varepsilon \rceil}|Y| \ge 2^{-194D/\varepsilon}|Y|$ for $0 < \varepsilon \le 1/160$) and the uncapped all-$D$ instance via `rUncappedRect2DFuel` $\varepsilon_2\ D = 2(D+1)\lceil 1/\varepsilon_2 \rceil + 1$ (`pow_rUncappedRect2DFuel_ge` proves $(2 : \mathbb{R})^{D+1} \le (1 + \varepsilon_2/2)^{\texttt{rUncappedRect2DFuel}\ \varepsilon_2\ D}$); the symbolic $\min(\cdot,\cdot)$ general-W formula of Lemma 4.3 is not separately formalized.
   - *Prune-Step Disjunction*: Both `cappedGoodRect3DLoop` / `cappedGoodRect3DLoopTracked` (`DyadicBucketingAndGoodCube.lean`) and `goodRectStep` / `goodRectStepTracked` (`Decomp2D.lean`) check `tag`$_1 =$ `2` $\vee$ `matDensity X' Y A` $\ge (1 + \varepsilon/2) \cdot$ `matDensity X Y A`, matching §4.3 Algorithm 4 Line 3 (`\label{line:findrect-rec1}`) and §5.2 Algorithm 5 (`cappedGoodCubeLoop` and `processGoodCubePiece`).
   - *3D `FindCube` Loop*: Lemma 5.3 (`\label{lem:cube-step}`) bounds the loop by $r \le R_{\text{cube}} := 1 + \lfloor 16/(\varepsilon\gamma) \rfloor$. Lean formalizes this via `rCubeFuel` $\varepsilon\ \gamma = 13 \cdot$ `Nat.ceil` $(1/(\varepsilon \cdot \gamma))$, with `rCubeFuel_le_`

`maxGoodCubeStepsReal` proving ($\texttt{rCubeFuel}\ \varepsilon\ \gamma : \mathbb{R}) \le 1 + 16/(\varepsilon \cdot \gamma)$ and `pow_rCubeFuel_ge` proving $(16 : \mathbb{R}) \le (1 + \varepsilon \cdot \gamma/4)^{\texttt{rCubeFuel}\ \varepsilon\ \gamma}$.

- *Remaining Loops*: `minDegreeLoop` / `minDegreeLoopTracked` uses well-founded structural recursion on $|U'|$ (`termination_by U'.card`), `processBlockTripleLoop` uses integer fuel $n \cdot n$ (`processBlockTripleLoop_spec`), `rDecompAFuel` / `rDecomp2DFuel` / `rUncappedDecomp2DFuel` fuel the 2D outer loops, and `rSStarFuel` (`rSStarFuel_ge_of_isBinaryMatrix`) fuels `sStarLoop`.

7. **Parameter d Floor-Division Identity in §6 Algorithm 6 Line 2 (`\label{line:tri-init}`)** : In `BMMGrid/TriangleAndBMM.lean`, `triDetParamD c k n` $= \max(320, \texttt{Nat.sqrt (Nat.log 2 n) / triDetC0Inv c k})$ equals $\max(320, \lfloor\sqrt{\lfloor\log_2 n\rfloor}/b_0\rfloor)$ for the integer $b_0 = c_0^{-1} =$ `triDetC0Inv c k` by the machine-checked identity `triDetParamD_eq_floor_div` ($\lfloor\lfloor x\rfloor/b_0\rfloor = \lfloor x/b_0\rfloor$ for $x \ge 0$ and integer $b_0 \ge 1$).

8. **`ValidCubePiece` & Entrywise Submatrix Dominance vs. §1.2 Definition 1.1 (`\label{def:pieces}`) & Theorem 1.1 (`\label{thm:main-decomp}`)** :

   - *Regular Pieces (`P.isRegularPiece = true`)*: In `ValidCubePiece` (`BMMGrid/Decomp3D.lean`), `P.isRegularPiece = true` asserts `P.X_k.Nonempty` $\wedge$ `P.Y_k.Nonempty` $\wedge$ `P.Z_k.Nonempty`, $(\varepsilon, 2, D)$-grid-regularity and ε-min-degree on A_k and B_k^{T}, and $\min(\mathbb{E}[A_k], \mathbb{E}[B_k]) > 2^{-D}$.
   - *Residual Pieces (`P.isRegularPiece = false`)*: Asserts $\min(\mathbb{E}[A_k], \mathbb{E}[B_k]) \le 2^{-D} \vee ((P.Y_k \times^s P.Z_k)$`.filter (fun` $p \Rightarrow P.B_k\ p.1\ p.2 = 1))$`.card` ≤ 1, without requiring non-emptiness of X_k, Y_k, Z_k so `thm_main_decomp` is total even on empty input sets (any piece with an empty vertex set has $A_k B_k = 0$ and potential 0).
   - *Entrywise Submatrix Dominance ($A_k \le A[X_k, Y_k]$ and $B_k \le B[Y_k, Z_k]$)*: Proved directly in `cappedABDecomposition_dominance` and included as an explicit conjunct of both `thm_3d_decomp_part2` and `thm_main_decomp` (`BMMGrid/Decomp3D.lean`): $\forall P \in$ `pieces`, $(\forall x, y,\ P.A_k\ x\ y \le$ `submatZeroExt` $P.X_k\ P.Y_k\ A\ x\ y) \wedge (\forall y, z,\ P.B_k\ y\ z \le$ `submatZeroExt` $P.Y_k\ P.Z_k\ B\ y\ z)$.

9. **Non-Formalized Consequences, Lower Bounds, and Constant-Trace Appendix**: §1.2 Corollary 1.3 (`\label{cor:consequences}`, downstream graph/circuit/geometry reductions), §4.4 Proposition 4.7 (`\label{prop:tightness}`, existential multi-round tree lower bound; Lean formalizes the single-step tightness witness in `rem_obstacles_single_step_tightness` and `rem_obstacles_multiblock_sift_tightness` in `Decomp2D.lean` conditional on supplied parameters / `PlantedMultiBlockConfig`; existence of such a configuration is not formalized), §5.1 Remark 5.2 (`\label{rem:why-one-fourth}`), and Appendix D (`\label{app:constants}`, numerical constant trace) are not formalized.

5 Class D — Implementation Choices Preserving the Proved Guarantees

The Lean definitions of Algorithms 1–6 (`minDegree` / `minDegreeTracked`, `concreteApproxGridNormV` / `concreteApproxGridNormVTracked`, `dyadicSift` / `dyadicSiftTracked`, `goodRectStep` / `aDecomposition` / `cappedGoodRect3D` / `cappedADecomp3D`, `cappedGoodCube` / `cappedABDecomposition` / `baseCase2Decomposition`, and `computableTriangleDetection` / `bmmFromTriDet`) adopt six implementation choices relative to the paper's pseudocode, each preserving the structural guarantees and the headline asymptotic complexity bounds; items 2, 4, and 5 also leave the functional outputs unchanged (`*_fst` lemmas), item 1 is a parameter choice (paper §8 (iv)) that is at least as accurate as the paper's, and items 3 and 6 are cost-only conventions cross-listed in Class B:

1. **Uniform `EstimateNorm` Accuracy Parameter (Parameter Choice, Not Control Flow)**: In `concreteApproxGridNormV` (`MinDegreeAndDyadicSift.lean`), Algorithm 2 (`EstimateNorm`) is invoked with uniform accuracy $\alpha' = (\varepsilon/32)2^{-3d}$ in place of Algorithm 3 Line 5's $\varepsilon\delta^3/32$ (a parameter choice rather than a control-flow change); since $\delta > 2^{-d}$ at every non-sparse sifting call, $\alpha' \le \varepsilon\delta^3/32$, preserving Lemma 4.2's certification and subrectangle density guarantees.

2. **Structural Recursion Fuel & Well-Founded Termination**: Each `while`-loop is totalized via an explicit `Nat.ceil` fuel counter (`rRectFuel`, `rRect2DFuel`, `rUncappedRect2DFuel`, `rDecompAFuel`, `rDecomp2DFuel`, `rUncappedDecomp2DFuel`, `rCubeFuel`, `rSStarFuel`), integer fuel n^2 (`processBlockTripleLoop`), or well-

founded structural recursion `termination_by U'.card` (`minDegreeLoop`), paired with sufficiency lemmas showing that the termination condition is reached before fuel exhaustion.

3. **Eager `EstimateNorm` Binding at `Sift` Entry in `dyadicSiftTracked` / `dyadicSiftConcreteTracked` (Charge, Not Control Flow)**: In `dyadicSiftTracked` (`MinDegreeAndDyadicSift.lean:1014-1055`, instantiated with `concreteApproxGridNormVTracked S X Y A d` in `dyadicSiftConcreteTracked` at `MinDegreeAndDyadicSift.lean:2299-2302`), `V_tracked` is bound via `CostM.bind` at procedure entry before the outer dyadic loop (whereas Algorithm 3 Line 5 invokes `EstimateNorm` only after the outer dyadic loop returns `none` and $X_{\geq \delta^2}$ is nonempty). This is purely cost-conservative: `dyadicSiftTracked_snd_le` upper-bounds the step cost by `V_tracked.2` $+ (4d+4)(|X|+1)(|Y|+1)$ and `dyadicSiftConcreteTracked_snd_le` upper-bounds it by $(|X|+|Y|+2)^2$(`concreteApproxGridNormCostFactor` $S.c\ S.k\ \varepsilon\ d + 2(d+1)$), while `dyadicSiftTracked_fst` and `dyadicSiftConcreteTracked_fst` prove that the returned value `.1` equals `dyadicSift` / `dyadicSiftConcrete` (which reads V only in the nonempty-$X_{\geq \delta^2}$ branch).

4. **Full-List Fold & Omitted Empty-Input Guard in Algorithm 6 (`TriangleDetect`)**: `triangleDetectionOnPieces` and `triangleDetectionOnPiecesTracked` (`TriangleAndBMM.lean`) omit Algorithm 6 Line 1's explicit empty-input guard (empty vertex sets already yield empty piece scans returning `false`) and fold boolean OR (`||`) over all pieces in $\mathcal{S}$ rather than terminating early upon the first piece where `detectOnPiece` returns `true`; the boolean output and worst-case cost bound are unchanged.

5. **Single-Witness Clearing per Detector Call in Lemma 6.1 BMM Reduction**: `processBlockTripleLoop` and `processBlockTripleLoopTracked` (`TriangleAndBMM.lean`) locate and clear one triangle witness (x, z) per `true` return of `computableTriangleDetection` (with integer fuel n^2 per block triple in `foldBlockTriples`, executing at most $\mathrm{nnz}(C) \leq n^2$ witness clears globally by `nnz_C_strictly_decreases_on_triangle`), preserving exact BMM output equality (`bmmFromTriDet_correct`) and the $O(n^3/2^{\Omega(\sqrt{\log n})})$ total cost bound (`lem_bmm_to_tridet_time_bound`).

6. **Iterative Peeling Charge in Algorithm 1 (`Prune` / `minDegree`; Charge, Not Control Flow)**: `minDegree` / `minDegreeTracked` (`MinDegreeAndDyadicSift.lean`) matches Algorithm 1's iterative "while $\exists u \ldots$ remove a minimum-degree vertex" loop directly (a Class B declared-charge choice, not a control-flow deviation), charging $O(|U||V| + |U|^2)$ across at most $|U|$ rounds (`minDegreeLoopTracked`, `minDegreeTracked_snd_le`) versus the paper's $O(|U||V|)$ bound (Lemma 3.2); the pruned subset (U', tag) and the $O(n^2)$ step envelope are unchanged.